

Perfect. This is where the actual architecture starts.

Again, we won't code first. We'll design it like an engineering team.

Module 2: Market Calendar

Purpose

The **Market Calendar** is responsible for determining **whether a given date is a valid trading day**.

It has **no knowledge of market timings (09:15-15:30)**.

It only answers questions about dates.

Responsibility

Only two responsibilities:

1. Check if a date is a trading day.
2. Find the next trading day.

That's all.

It should **never**:

- Check current time
 - Sleep
 - Publish events
 - Know market open/close timings
 - Know anything about users or strategies
-

Inputs

Date

Example:

2026-08-05

Outputs

For `is_trading_day()`:

True

or

False

For `get_next_trading_day()`:

2026-08-06

Why is this module required?

Without it, the Market Session Manager would contain logic like:

Python

Run

```
if weekday == 5:
    ...

if weekday == 6:
    ...

if holiday:
    ...
```

Now imagine NSE changes a holiday.

You'd have to modify the Market Session Manager.

Instead:

Market Session Manager

↓

Market Calendar

↓

True / False

The responsibilities stay separated.

Public API

I would expose only these methods:

Python

Run

```
class MarketCalendar:

    def is_trading_day(self, date) -> bool:
        ...

    def get_next_trading_day(self, date):
        ...
```

Nothing more for Version 1.

Internal Logic (Version 1)

Initially, keep it simple.

Trading Days:

Monday

Tuesday

Wednesday

Thursday

Friday

Non-trading:

Saturday

Sunday

Ignore holidays for now.

Later, you'll add:

NSE Holiday List

without changing the Session Manager.

Internal Workflow

is_trading_day()

Date

↓

Weekday?

↓

Saturday / Sunday?

↓

Yes → False

↓

No → True

get_next_trading_day()

Current Date

↓

+1 Day

↓

Trading Day?

↓

No

↓

+1 Day

↓

Trading Day?

↓

Yes

↓

Return Date

Dependencies

Version 1:

- Python `datetime`

Only.

No database.

No internet.

No broker.

Failure Cases

Consider these:

- Invalid date object
- `None` passed as input
- Unsupported date type

Raise an exception rather than silently failing.

Unit Tests

These are the tests I'd write.

Test 1

Monday

↓

True

Test 2

Friday

↓

True

Test 3

Saturday

↓

False

Test 4

Sunday

↓

False

Test 5

Friday

↓

Next Trading Day

↓

Monday

Test 6

Wednesday

↓

Next Trading Day

↓

Thursday

Folder Structure

market_session/

```
|— market_state.py
|— market_calendar.py
└— tests/
    └— test_market_calendar.py
```

One improvement before coding

Even though Version 1 ignores holidays, **design the class so holidays can be added later without changing its public API.**

For example, internally you might later have:

Python

Run

```
self.holidays = {
    date(2026, 1, 26),
    date(2026, 8, 15),
    ...
}
```

Then `is_trading_day()` simply checks:

1. Is it Saturday or Sunday?
2. Is it in the holiday set?

The **Session Manager** and **Market Session Manager** won't need any changes because they already call:

Python

Run

```
calendar.is_trading_day(date)
```

This is an example of designing for future extension while keeping today's implementation simple.

After this design is finalized, we'll implement `market_calendar.py` together.